Comprende

Checkpoint Metacognitivo

¿Puedes explicar en tus propias palabras por qué usar la cabeza de la lista enlazada como tope hace que push y pop sean O(1)?

Como la ubicación de la cabeza ya es conocida no es necesario buscarla, por lo que las operaciones siempre serán las mismas sin importar el tamaño de entrada.

Si alguien te pregunta "¿para qué sirve una pila y dame dos ejemplos reales?", ¿puedes responder sin consultar estas notas? Si no, relee la sección de aplicaciones.

Sí, la pila sirve para registrar una especie de historial de datos. Un ejemplo sería el historial del navegador, que te permite ir a la página web anterior. Otro ejemplo sería el historial de cambios de un procesador de textos, el cual te permite deshacer cambios.

¿Cuál es la diferencia funcional entre peek() y pop()? ¿En qué situación usarías uno vs el otro en el verificador de paréntesis?

Su principal diferencia es que peek() solo se usa para consultar el dato que está en la cima, mientras que pop() lo elimina. Si solo quiero consultar el dato usaría peek y si quiero eliminarlo usaría pop.

Alerta IA: Si le pediste a la IA que "te explicara las pilas", ¿puedes reproducir esa explicación con tus propias palabras? Si no, vuelve a preguntar pidiendo que use analogías más simples.

No se lo pedí. Pero esencialmente la pila puede entenderse por su significado literal. Una pila es un amontonamiento de cosas, o un conjunto de cosas puestas una sobre otra.

Aplica

Reto 1

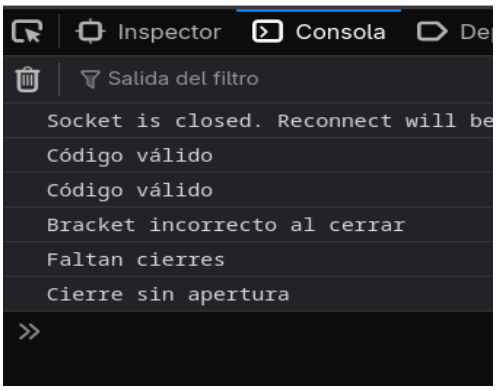
Véase el archivo de código en GitHub. Adjunto la salida real que produce el código:

Reto 2

Una simple variable que cuente los signos de agrupación no sería suficiente. El contador solo contaría los signos de apertura que se encuentre, pero no tendría en cuenta su orden ni su tipo. Es por eso que es esencial usar una pila que almacene los signos ordenadamente, lo que nos permite compararlos.

```javascript
verificarBalanceo(codigoFuente) {
    const pila = new Pila();
    let esValido;

    for (let i = 0; i < codigoFuente.length; i++) {

        if (codigoFuente[i] === "(" || codigoFuente[i] === "{" || codigoFuente[i] === "[") {
            pila.push(codigoFuente[i]);
        }

        if (codigoFuente[i] === ")" || codigoFuente[i] === "}" || codigoFuente[i] === "]") {

            if (pila.estaVacia()) {
                esValido = false;
                return "Cierre sin apertura";
            }

            let ultimo = pila.pop();

            if ( (ultimo === "(" && codigoFuente[i] !== ")") ||
                (ultimo === "[" && codigoFuente[i] !== "]") ||
                (ultimo === "{" && codigoFuente[i] !== "}") )
            {
                esValido = false;
                return "Bracket incorrecto al cerrar";
            }
        }
    }

    esValido = true;
    return !pila.estaVacia() ? "Faltan cierres" : "Código válido";
}
```

```javascript
const verificar = new EditorCodeLeap();
console.log( verificar.verificarBalanceo("({}[])") )
console.log( verificar.verificarBalanceo("({[]})") )
console.log( verificar.verificarBalanceo("({[}])") )
console.log( verificar.verificarBalanceo("((") )
console.log( verificar.verificarBalanceo("))") )
```



¿Puedes explicar sin mirar el código por qué se necesitan DOS pilas para implementar Deshacer + Rehacer, y no una sola?

Para tener el historial de los cambios y además el historial de los cambios deshechos. Si solo se tuviera una pila se podrían deshacer cambios, pero no rehacerlos.

¿Puedes trazar a mano el estado de la pila con la cadena `"{[}]"` para demostrar que el verificador la rechaza correctamente?

Quedaría así:

[

{

Se comprueba “[“ == “}”. Devuelve falso.

Reflexiona

Reto 3

Me parece que lo mejor sería utilizar un arreglo dinámico. Para operaciones básicas que tienen que ser muy veloces el arreglo se beneficia porque sus elementos se guardan posiciones de memoria contigua. Además, la lista enlazada tiene una sobrecarga de memoria porque necesita de un puntero para enlazar los nodos.

En el caso del editor de código CodeLeap, teniendo en cuenta la superioridad en velocidad y el ahorro en memoria que tiene el arreglo dinámico, yo usaría a este último en lugar de la lista enlazada. Especialmente porque se supone que la implementación en código de la pila con un arreglo dinámico sería más simple y tendría menos líneas.

Mi criterio personal para elegir una pila en lugar de otra estructura de datos sería preguntarme a mí mismo lo siguiente: ¿Lo último que inserté es lo que debo recuperar primero? ¿Solo necesito acceder al primer elemento? Si ambas preguntas tienen un sí como respuesta entonces sé que lo mejor será usar una pila. En caso contrario, debería explorar otras opciones.

Explica en una oración: ¿por qué la pila de llamadas del sistema operativo usa exactamente el principio LIFO?

Porque las funciones tienen una jerarquía ya que pueden estar anidados, por eso debe ejecutarse primero la última llamada que haya entrado.

¿Podrías defender ante un colega tu elección de implementación (lista enlazada vs arreglo) para la pila de CodeLeap? ¿Qué criterio usaste?

Sí, simplemente les diría que el arreglo dinámico usa menos memoria y el procesador actúa más rápido porque los elementos están en posiciones contiguas de memoria.

Valida

¿Pasaron TODOS los casos de prueba del checklist? Si alguno falló, ¿ya identificaste la línea de código específica que lo causa?

Me enorgullece decir que sí pasaron todos.

¿Puedes explicar por qué el caso `"({[}])"` es inválido sin ejecutar el código — solo razonando con el estado de la pila en cada paso?

El primer signo de cierre se compara con el último signo de apertura, no coinciden, así que es inválido.